

AI IN ACTION-P1 · NGÀY 9

Multi-Agent & Kết Nối Hệ Thống

Từ một agent tốt sang một hệ thống rõ vai trò

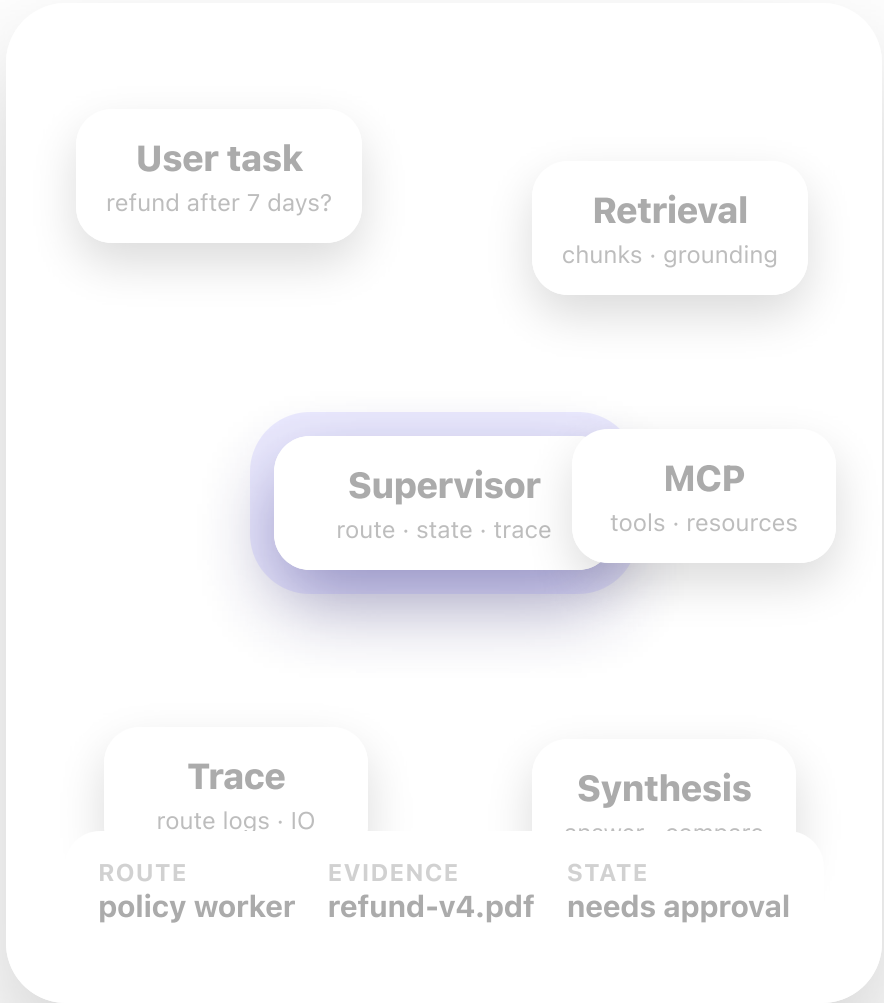
Supervisor

Workers

MCP / A2A

Trace

Nối tiếp trực tiếp từ artifact Day 08 · use case xuyên ngày: trợ lý nội bộ cho khối CS + IT Helpdesk



Instructor

Giữ layout từ mẫu; nội dung buổi học bám YAML Day 09



[Tên giảng viên]

[Chức danh / vai trò]

Audience: Học viên đã có artifact RAG từ Day 08 hoặc hệ retrieval-answer cơ bản.

Teaching goal: rõ vai, dễ trace, dễ mở rộng

Case study xuyên ngày: CS + IT Helpdesk nội bộ

Bridge: Day 08 artifact → Day 09 orchestration → Day 10 observability

Focus hôm nay

Supervisor · Workers · MCP · A2A · LangGraph · Trace

Mục tiêu học tập & đầu ra cuối ngày

Học pattern, contract, MCP, LangGraph; build supervisor + workers + trace

**4 GIỜ LÝ
THUYẾT**

overload → pattern → supervisor → MCP/A2A →
LangGraph

**4 GIỜ THỰC
HÀNH**

graph → workers → MCP capability → trace &
docs

LEARN

Mental model cần nắm

- Single-agent quá tải ở đâu
- Pattern: supervisor-worker, pipeline, debate, hierarchical
- MCP, A2A, LangGraph, HITL

BUILD

Artifact cuối ngày

- supervisor / orchestrator chạy được
- 2–3 workers rõ vai trò
- 1 MCP capability + trace/log toàn luồng

2–3

workers rõ vai

1

MCP capability

1

trace dễ đọc

Nhắc lại trước khi tách hệ

Day 09 không bắt đầu lại từ đầu; nó bọc orchestration lên artifact Day 08

CASE STUDY XUYỀN 3 NGÀY

Trợ lý nội bộ cho khối CS + IT Helpdesk

refund policy 7 ngày access approval workflow P1 ticket SLA knowledge base có version

Day 08

RAG pipeline

retrieve · ground · evaluate

refund policy · P1 SLA · access workflow

Day 09

Orchestration

supervisor · workers · MCP · trace

rõ vai · dễ trace · dễ mở rộng

Day 10

Data + observability

pipeline · quality · lineage · monitoring

giữ hệ sống được ngoài đời thật

Thông điệp chính

Day 09 không “thần thánh hoá” multi-agent; chỉ tách vai khi một agent đang gánh quá nhiều trách nhiệm.

Một agent rất giỏi. Nhưng bài toán đã quá lớn cho 1 agent.

Plan task, retrieve, call tools, synthesize, monitor + retry — tất cả dồn vào một nơi



1 **Một nơi ôm mọi quyết định**
plan · retrieve · tool · answer · retry

2 **Khó debug**
không rõ lỗi do route, input hay tool

3 **Khó mở rộng**
mỗi tool mới làm prompt phình to hơn

Câu hỏi đúng: có đang ép 1 agent gánh quá nhiều vai trò không?

Hoạt động 1 — Agent overload map

Vẽ lại agent hiện tại và đánh dấu chỗ quá tải

8 phút nhóm • 2 phút chia sẻ

TASK

- liệt kê plan / retrieve / tool / synthesize / retry
- đánh dấu chỗ khó debug nhất
- chọn 1 vai trò nên tách trước

OUTPUT

1 sơ đồ monolith agent
+ 1 đề xuất tách vai đầu tiên

TEMPLATE OVERLOAD MAP



Gợi ý nhanh: nếu thêm 1 tool mới làm prompt dài lên đáng kể, đó thường là tín hiệu nên tách vai.

Khi nào single-agent bắt đầu quá tải?

4 giới hạn cốt lõi cần nhớ trước khi quyết định tách hệ thống



Context bottleneck

prompt ngày càng phình to



Specialization trade-off

một prompt phải làm quá nhiều việc



Parallelism hạn chế

việc độc lập vẫn bị serial



Reliability yếu

route sai đầu luồng kéo lệch cả flow

Đừng dùng multi-agent chỉ vì thấy “ngẫu”. Hãy dùng khi nó giảm coupling và tăng khả năng quan sát.

4 pattern multi-agent phổ biến

Supervisor-worker, pipeline, debate, hierarchical

Supervisor-worker

route rõ, dễ trace



warning: supervisor tệ = bottleneck

Pipeline

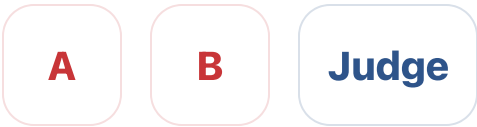
flow tuyến tính, dễ test



warning: kém linh hoạt

Debate

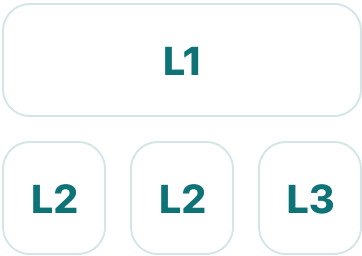
nhiều góc nhìn, giảm blind spot



warning: tốn cost và latency

Hierarchical

mở rộng tốt theo domain



warning: debug khó hơn

Hôm nay đi sâu

Supervisor-worker vì dễ nối trực tiếp từ artifact Day 08 và đủ thực dụng cho lab.

Hoạt động 2 — Chọn pattern cho 3 use case

Không phải bài toán nào cũng cần cùng một pattern

USE CASE A

route câu hỏi sang đúng skill

pattern nào?

khi nhiều domain / skill

USE CASE B

nhiều bước cố định, ít thay đổi

pattern nào?

khi flow tuyến tính

USE CASE C

cần nhiều góc nhìn để giảm blind spot

pattern nào?

khi cần challenge / phản biện

Chọn pattern

supervisor-worker **pipeline** **debate**

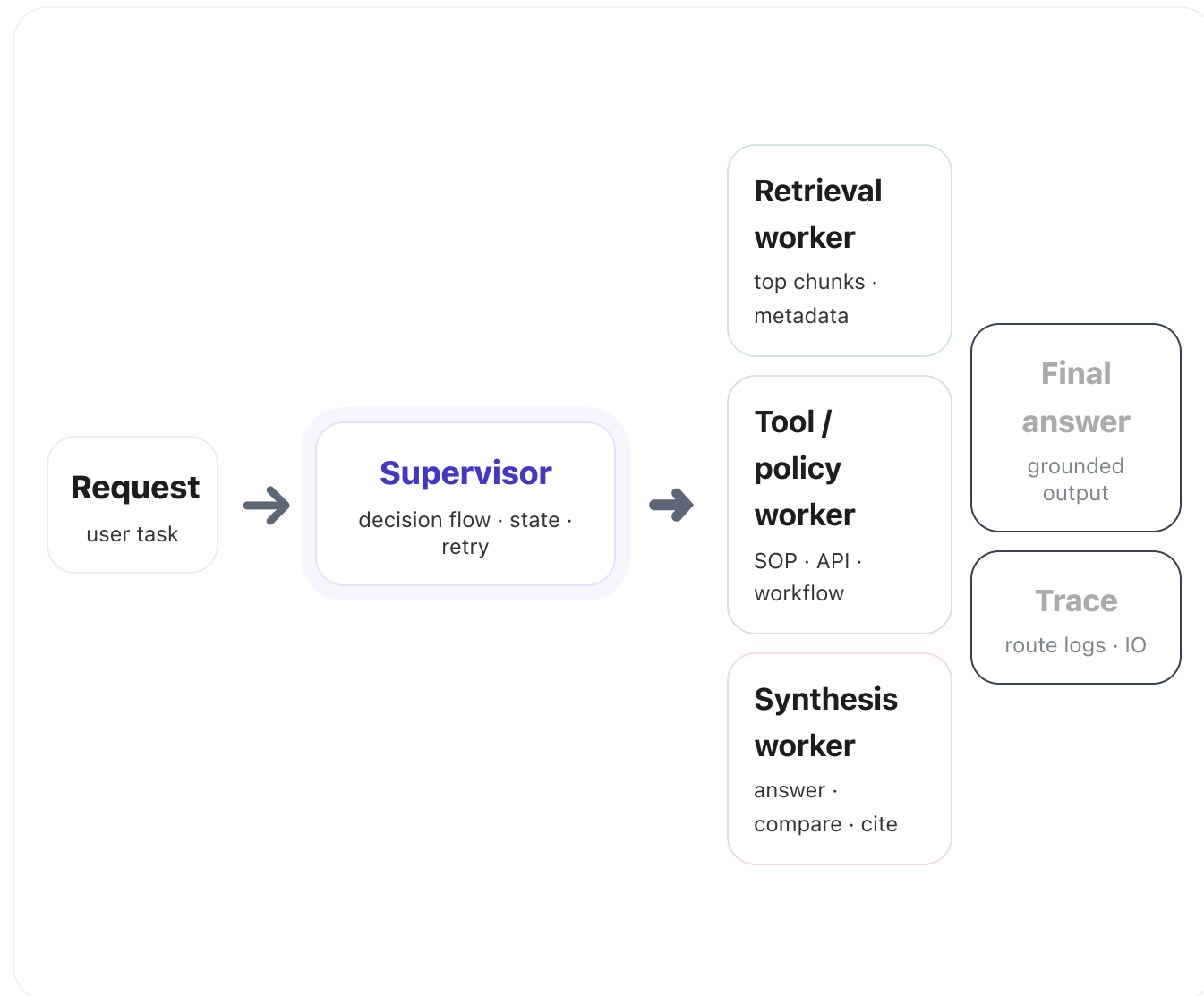
hierarchical

Ghi 1 rủi ro

Vì sao không chọn pattern còn lại?

Vì sao chọn supervisor-worker?

Pattern dễ dạy, dễ build, dễ trace và dễ nối từ Day 08



Supervisor giữ

- route theo task / risk / domain
- state chung của flow
- khi nào cần HITL hoặc retry

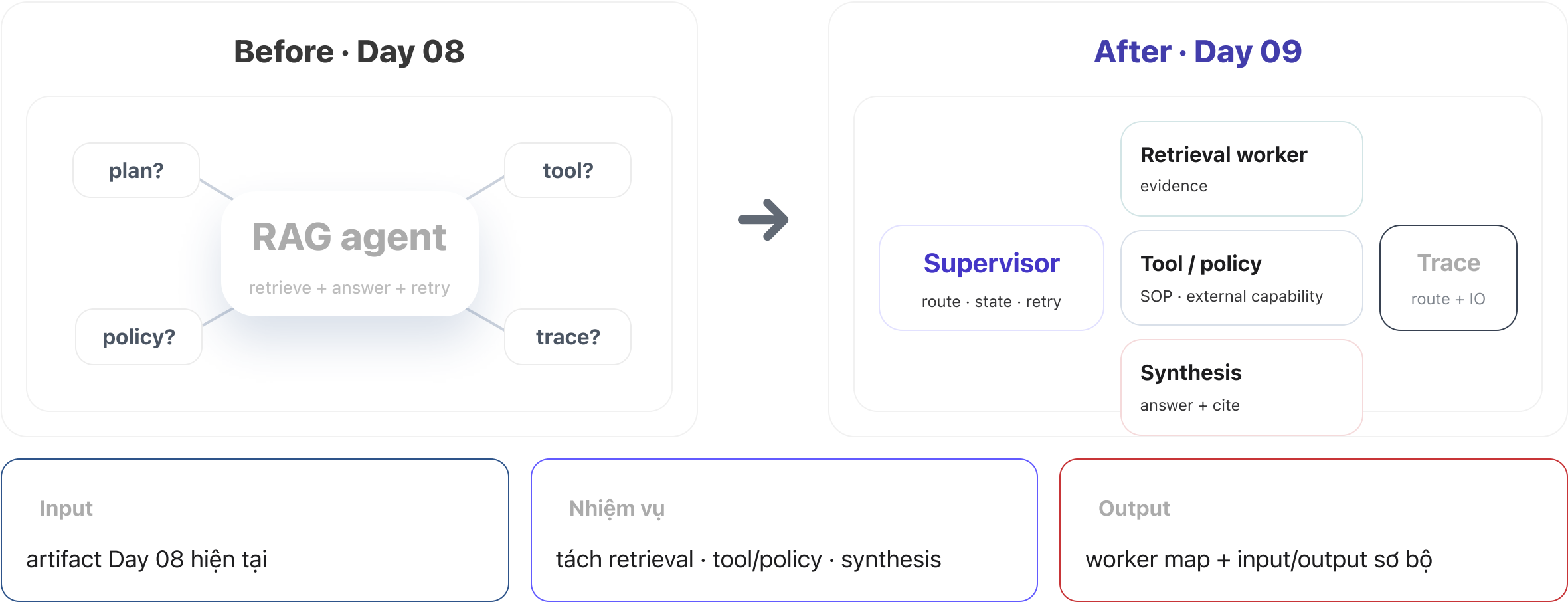
Worker giữ

- năng lực hẹp, input/output rõ
- càng stateless càng dễ test
- thay worker mà ít ảnh hưởng hệ

Rule: supervisor không cần “thông minh hơn tất cả”; chỉ cần phân việc đủ rõ và gom đầu ra đủ sạch.

Hoạt động 3 – Tách artifact Day 08 thành workers

Refactor từ một RAG agent sang nhiều vai trò



Worker contract: rõ input, rõ output, rõ lỗi

Shared state giúp điều phối; message contract giúp giao tiếp không nhập nhằng

JSON CONTRACT TỐI THIỂU

```
{
  "task": "retrieve_evidence",
  "context": "refund policy after 2025",
  "constraints": ["internal_only"],
  "expected_output": "top_3_chunks",
  "error": {"code": "...", "reason": "..."}
}
```

testable traceable replaceable

Shared state

phù hợp cho graph/orchestrator giữ state chung của run

Message contract

phù hợp cho giao việc rõ ràng giữa supervisor và worker

Rule đơn giản

Supervisor giữ quyết định luồng. Worker giữ domain skill. Contract là ranh giới giữa hai bên.

Điều phối Động vs. Tĩnh (Router vs. Chain)

Khi nào cần sự linh hoạt của Router thay vì bước cố định của Chain

Static Chain (Tĩnh)

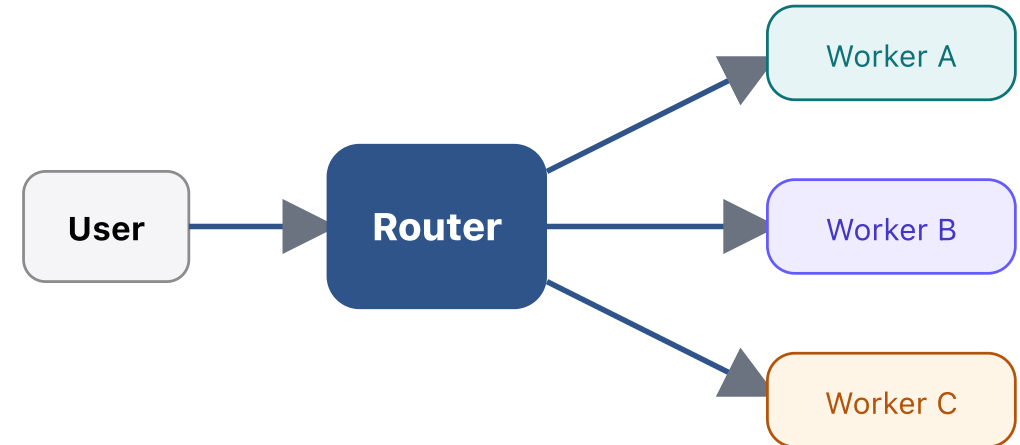
Các bước A -> B -> C cố định.

Dùng khi: Quy trình chuẩn (SOP) không đổi, đầu ra bước trước là đầu vào bước sau.

Dynamic Router (Động)

Dựa vào input để chọn bước tiếp theo.

Dùng khi: Yêu cầu người dùng đa dạng, không thể đoán trước luồng đi (IT Helpdesk).



"Bộ não" của Supervisor: Prompting để điều phối

Cách viết System Prompt để Supervisor không "làm hộ" Worker

SYSTEM PROMPT PHÂN VAI

```
# Vai trò: Điều phối viên Helpdesk
1. CHỈ phân loại và gọi worker. KHÔNG tự trả lời kiến thức chuyên môn.
2. Nếu thiếu thông tin, gọi 'Clarifier Worker'.
3. Danh sách workers:
  - 'RefundBot': xử lý chính sách hoàn tiền.
  - 'TechBot': xử lý lỗi kết nối/account.
  - 'Escalation': chuyển cho người thật.
```

Lỗi thường gặp

Supervisor quá thông minh nên "tự biên tự diễn", bỏ qua worker và dẫn đến ảo giác (hallucination) vì không dùng evidence từ worker.

Kỹ thuật: Cung cấp mô tả worker thật ngắn gọn và ranh giới rõ ràng trong prompt.

Quản lý State: Theo dõi tiến độ task

Cách Supervisor "nhớ" những gì đã làm để không bị lặp vòng

History

Các bước đã đi qua

Pending

Các task chưa xong

Decision

Lý do chọn worker

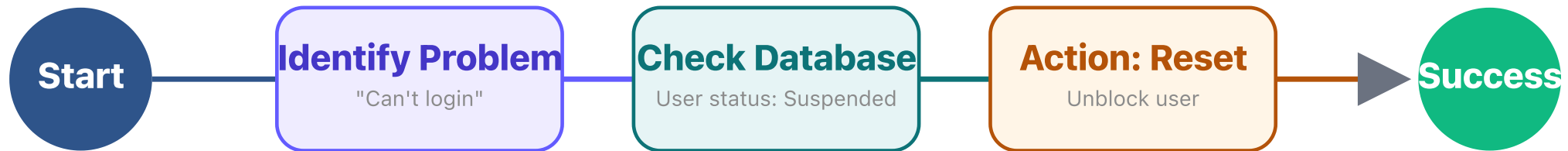
STATE OBJECT SCHEMA

```
{ "current_task": "...", "status": "in_progress", "history": ["route: tech", "tool: ping_server", "result: timeout"],  
  "next_worker": "escalation" }
```

State là "bảng chấm công" của hệ thống, giúp Supervisor biết dừng lại khi task đã hoàn thành hoặc thất bại.

Luồng Troubleshooting thực tế

Ví dụ luồng đi của Supervisor khi xử lý lỗi đăng nhập



Trong mỗi bước, Supervisor là người quyết định: "Tiếp tục gọi DB worker" hay "Xong rồi, báo user".

MCP architecture: agent cắm vào năng lực bên ngoài theo chuẩn chung

Agent/MCP client -> MCP server -> tools/resources/prompts

ANALOGY

MCP = ổ cắm chuẩn

Agent lấy năng lực bên ngoài qua một surface chung, thay vì hard-code từng integration.

Tools

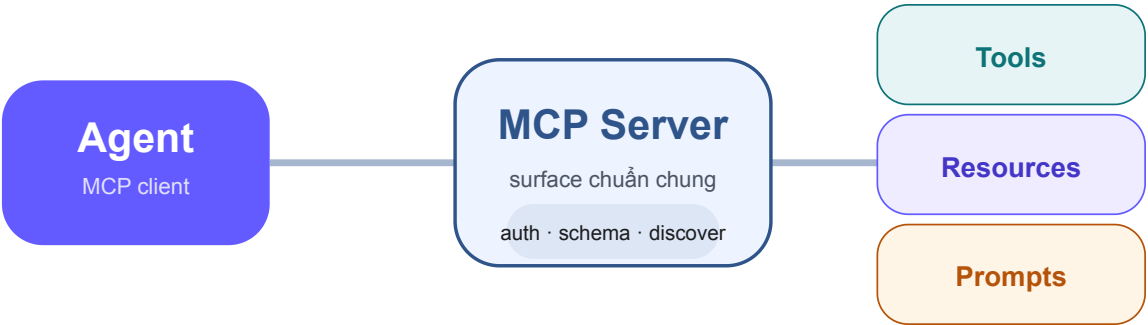
search API ·
ticket action

Resources

schema ·
config · docs

Prompts

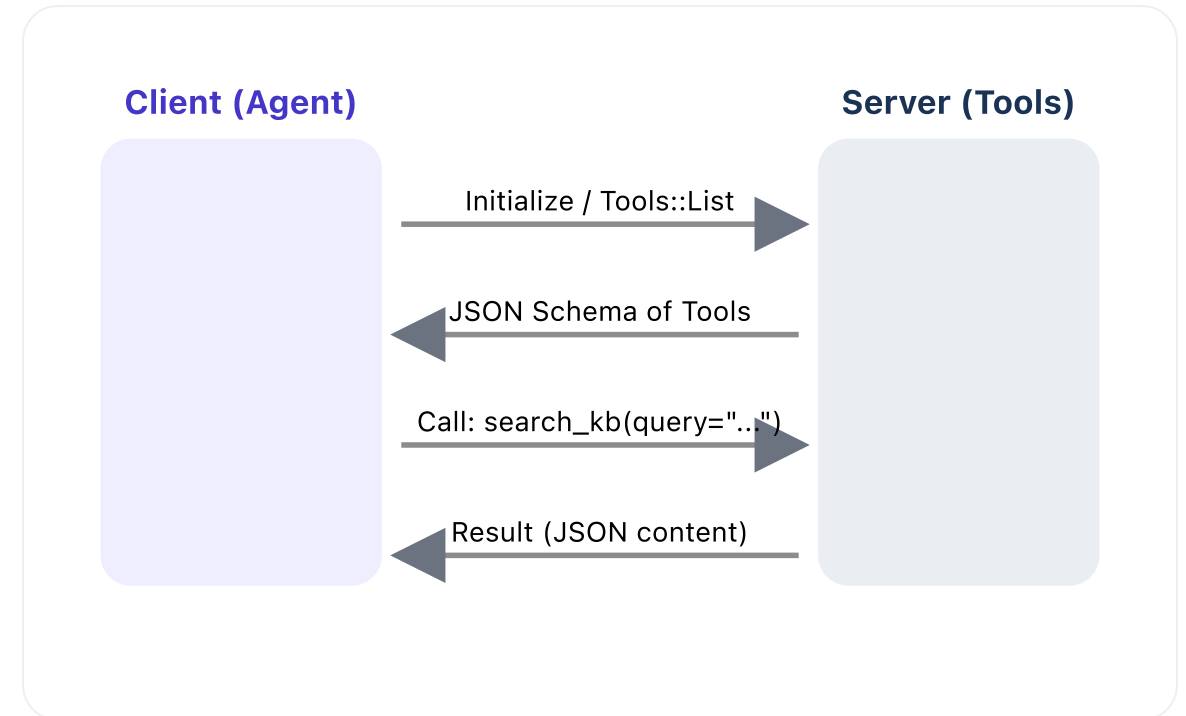
template /
policy surface



Vòng đời kết nối MCP (Discovery & Lifecycle)

Agent tìm thấy năng lực mới như thế nào?

- **1. Initialization:** Client (Agent) kết nối tới Server.
- **2. Discovery:** Server liệt kê các **tools**, **resources** và **prompts** có sẵn.
- **3. Execution:** Agent gọi tool cụ thể với tham số JSON.
- **4. Sampling:** Server có thể yêu cầu Agent thực hiện một hành động (Agent-as-a-service).



Kết nối công cụ thật: Zendesk & Jira

Dùng MCP để biến API thành năng lực của Agent

USE CASE

Tự động hóa Ticket Escalation

Zendesk Server

`get_ticket_details` `update_status` `add_internal_note`

Jira Server

`create_issue` `assign_developer` `comment_on_bug`

Thay vì Agent phải "hiểu" API của Zendesk, MCP Server dịch API đó thành các **Tool Definition** mà LLM có thể gọi dễ dàng.

3 trụ cột của MCP: Tool, Resource, Prompt

Phân biệt và cách dùng đúng trong hệ thống Agent

Tools (Hành động)

"Làm" một việc gì đó có side-effect: tạo File, gửi Mail, chạy SQL.

Resources (Dữ liệu)

"Đọc" dữ liệu tĩnh hoặc log: bảng Schema DB, nội dung File PDF, System Log.

Prompts (Mẫu)

"Hướng dẫn" định sẵn cho Agent: mẫu trả lời Email, checklist bảo mật.

Bảo mật & Ranh giới ủy quyền

Agent không nên có quyền "full root" trên server

Least Privilege (Quyền tối thiểu)

Chỉ cấp quyền gọi tool đọc Ticket, không cho quyền xóa toàn bộ Database.

User Approval (Xác thực người dùng)

Các hành động nguy hiểm (như Refund) PHẢI có sự đồng ý của con người trong luồng.

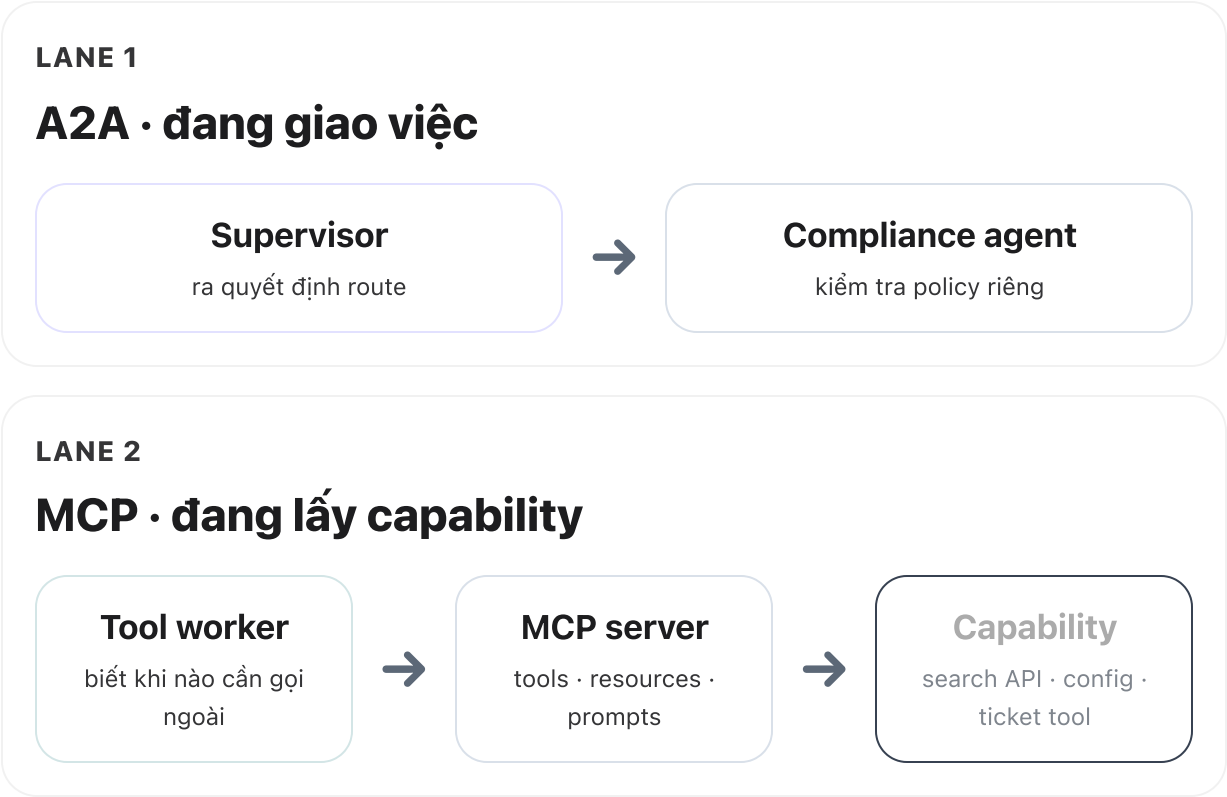
Sandboxing

MCP Server nên chạy trong container/môi trường cô lập để Agent không phá hỏng hệ thống.

Kịch bản rủi ro: Prompt injection khiến Agent cố tình gọi tool xóa dữ liệu. MCP Server phải là lớp phòng thủ cuối cùng kiểm tra quyền thực sự của Agent.

A2A = giao việc cho agent khác · MCP = lấy năng lực từ bên ngoài

Kèm sync vs async và boundary cần nhớ trước khi chạm lab



A2A

agent → agent khi đang **giao việc** cho một vai trò khác

MCP

agent → capability khi đang **lấy tool / resource / prompt**

Sync vs async

sync cho task ngắn / rõ; async cho task dài, batch hoặc handoff

Boundary

auth scope, least privilege, approval trước action có side effect

Hoạt động 4 — Phân biệt MCP với A2A

Cùng là gọi ra ngoài, nhưng gọi tool khác với giao việc cho agent khác

8 phút phân loại · 2 phút chốt rule

GỢI Ý

Hỏi: đang lấy năng lực hay đang giao việc?

OUTPUT

1 bảng phân loại
+ 1 rule of thumb

MCP

tool / resource / prompt

A2A

agent giao việc cho agent

tool worker gọi search API

supervisor nhờ compliance agent kiểm tra policy

supervisor đọc config/resource từ external system

synthesis worker nhờ red-team agent challenge answer

tool worker gọi ticket action

supervisor giao escalation cho agent khác

Rule of thumb: **tool** hay agent?

Các mô hình cộng tác Agent (A2A Patterns)

Handoff, Cooperative Search & Concurrent Debate

Handover

Sequential

Agent A bàn giao task cho Agent B khi xong phần việc của mình.

Cooperative Search

Parallel

Nhiều Agent cùng tìm kiếm ở các nguồn khác nhau để gom dữ liệu nhanh.

Joint Decision

Consensus

Cần sự đồng ý của cả 2 Agent (ví dụ: Tech + Compliance) trước khi trả lời.

Self-Correction

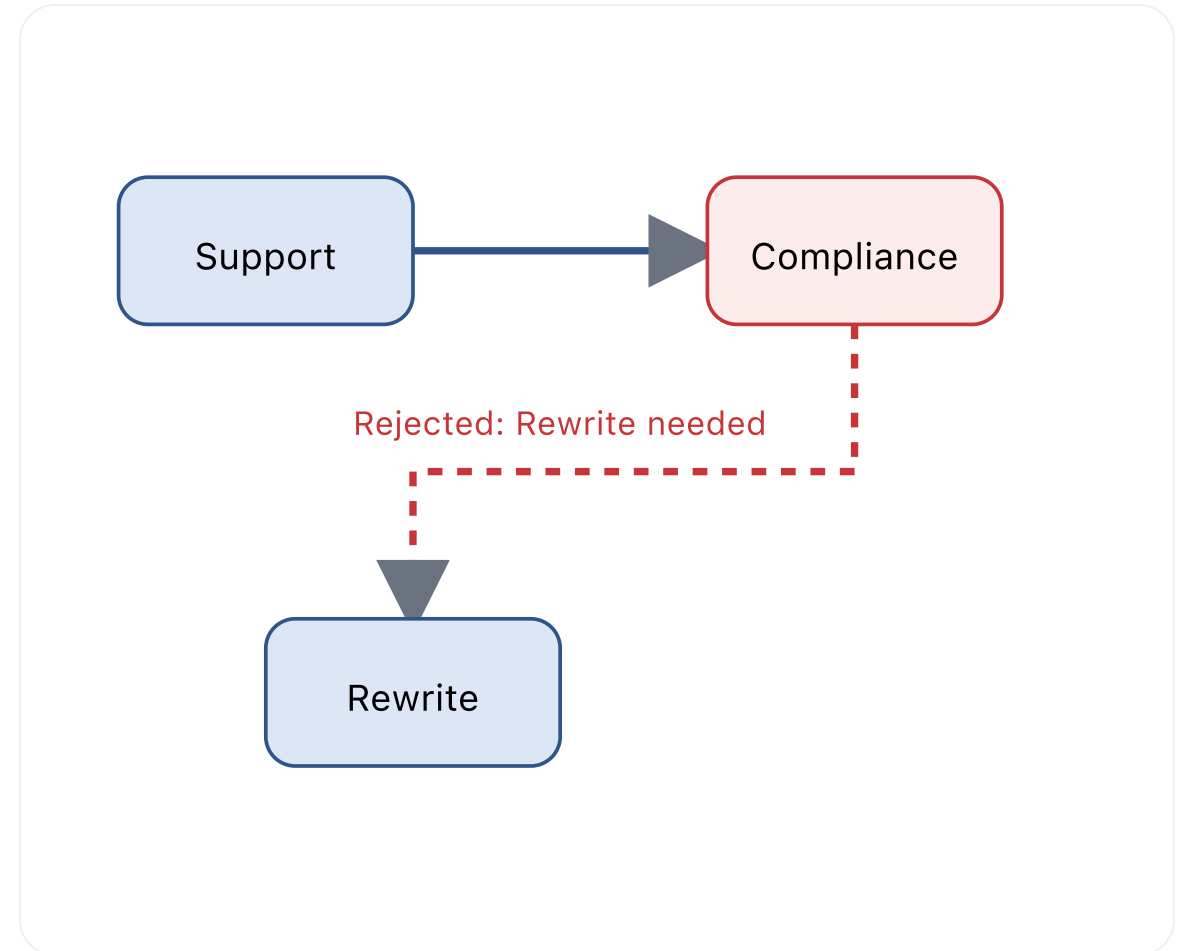
Feedback Loop

Agent A viết, Agent B check và yêu cầu A sửa lại nếu có lỗi.

Case Study: Compliance Guardrail

Dùng A2A để kiểm soát rủi ro pháp lý và chính sách

- 1 **Support Agent** soạn thảo câu trả lời dựa trên Knowledge Base.
- 2 **Compliance Agent** nhận bản nháp, đối chiếu với danh sách "Blacklist" (từ cấm, cam kết sai...).
- 3 Nếu có lỗi, Compliance Agent yêu cầu Support Agent sửa lại hoặc Escalation cho Manager.



Xử lý mâu thuẫn giữa các Agent

Khi "ông nói gà, bà nói vịt" — giải quyết như thế nào?

Conflict (Mâu thuẫn)

Agent A bảo có thể refund, Agent B bảo khách này vi phạm chính sách bảo mật nên không được.

Solution: Judge Agent

Dùng một Agent thứ 3 có Prompt "Phân xử" để cân nhắc bằng chứng của cả 2 bên.

Solution: Hierarchy

Quy định Agent nào có "trọng số" cao hơn (ví dụ: Compliance Agent luôn thắng Support Agent).

Last Resort: HITL

Nếu vẫn mâu thuẫn, chuyển cho người thật (Manual Intervention).

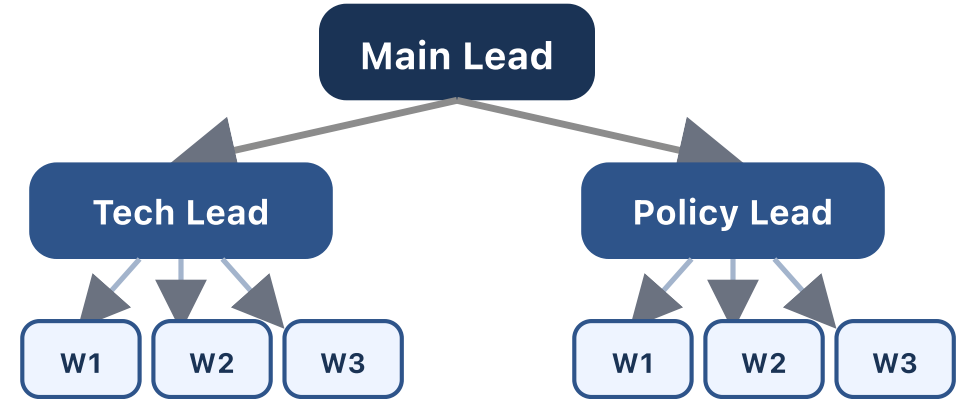
Cấu trúc phân tầng (Hierarchical Teams)

Mở rộng hệ thống bằng cách biến Supervisor thành một Worker

Khái niệm

Thay vì một Supervisor quản lý 100 workers, chúng ta tạo các **Team Leads**. Mỗi Team Lead là một Supervisor nhỏ quản lý 3-5 workers.

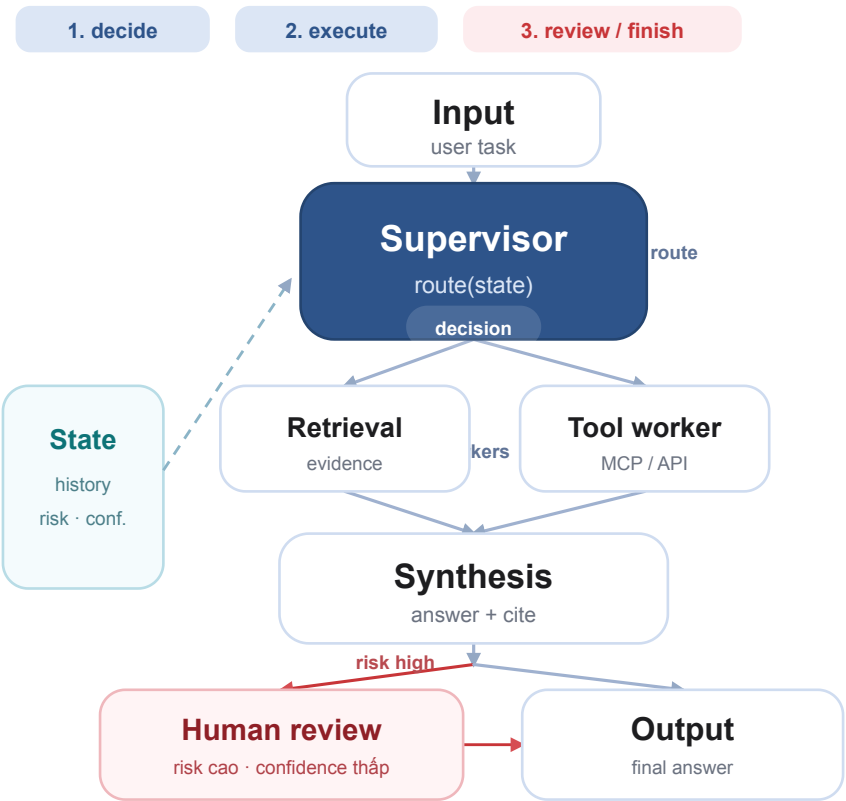
Lợi ích: Giảm tải context cho Supervisor tổng, dễ debug theo module (IT Team, Sales Team, HR Team).



Mỗi lead quản 3–5 workers
(W = worker / specialist)

LangGraph: node, edge, state, route, HITL

Routing logic nên tường minh trong graph, không nên chôn trong prompt khổng lồ



node

ai làm

edge

đi đâu tiếp

state

hệ đang biết gì

hitl

risk cao / confidence thấp

ROUTE(STATE)

```
def route(state):
    if state["risk_high"]:
        return "human_review"
    if state["needs_tool"]:
        return "tool_worker"
    return "retrieval_worker"
```

Thông điệp chính

Routing logic nên lộ ra thành graph/code; đừng chôn nó trong một prompt điều phối khổng lồ.

Tính kiên định & Checkpointing trong LangGraph

Làm sao hệ thống "nhớ" được buổi hội thoại ngày hôm qua?

Thread ID

Mã định danh buổi chat

Checkpointer

Lưu state vào Database

Resume

Tiếp tục flow từ điểm dừng

LANGGRAPH MEMORY SETUP

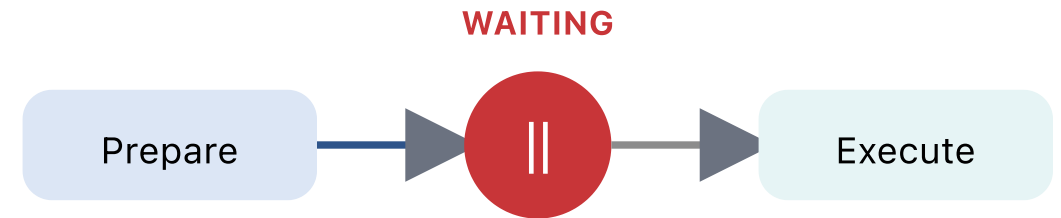
```
memory = SqliteSaver.from_conn_string(":memory:")
graph = builder.compile(checkpointer=memory)
# Chạy graph với thread_id cụ thể
config = {"configurable": {"thread_id": "user_123"}}
graph.invoke(input, config)
```

Checkpointing cho phép "Time Travel": Bạn có thể quay lại bất kỳ thời điểm nào trong quá khứ của Graph để sửa lỗi hoặc chạy thử kịch bản khác.

Human-in-the-loop (HITL) & Phê duyệt

Tạm dừng Graph để chờ sự can thiệp của con người

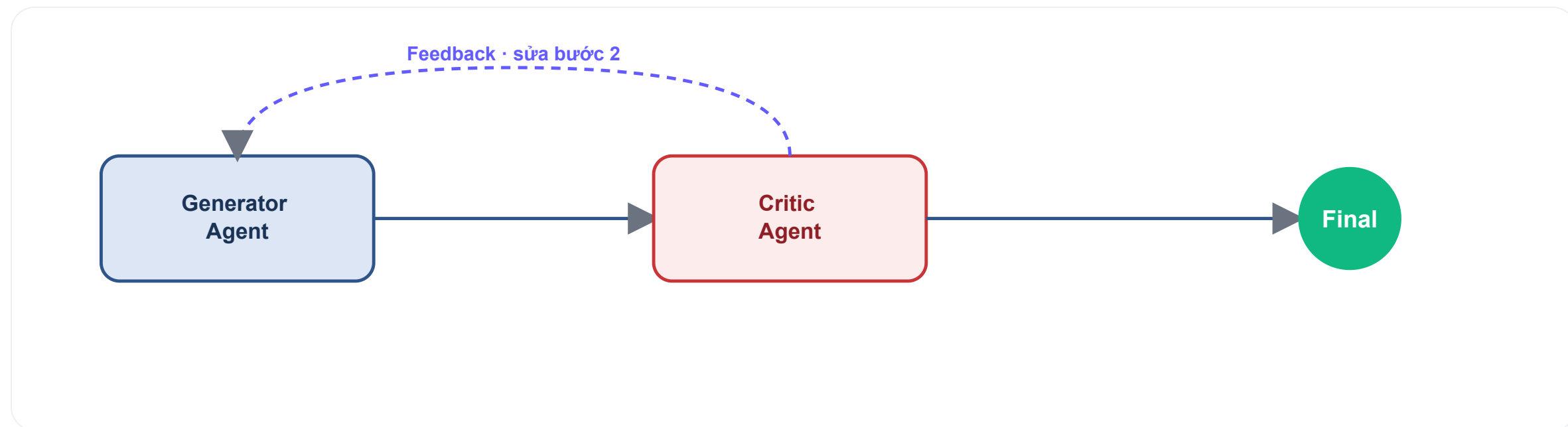
- ! **Breakpoint:** Graph tự động dừng trước một Node nhạy cảm (ví dụ: ``send_refund_email``).
- ? **Review:** Manager xem lại lý do Refund và bằng chứng Agent đã gom.
- o **Action:** Manager bấm "Approve" -> Graph tiếp tục chạy Node tiếp theo.



Đảm bảo an toàn cho các tác vụ thay đổi dữ liệu hoặc tài chính.

Vòng lặp tự sửa lỗi (Self-Correction)

Agent tự soi gương và sửa lỗi trước khi trả kết quả

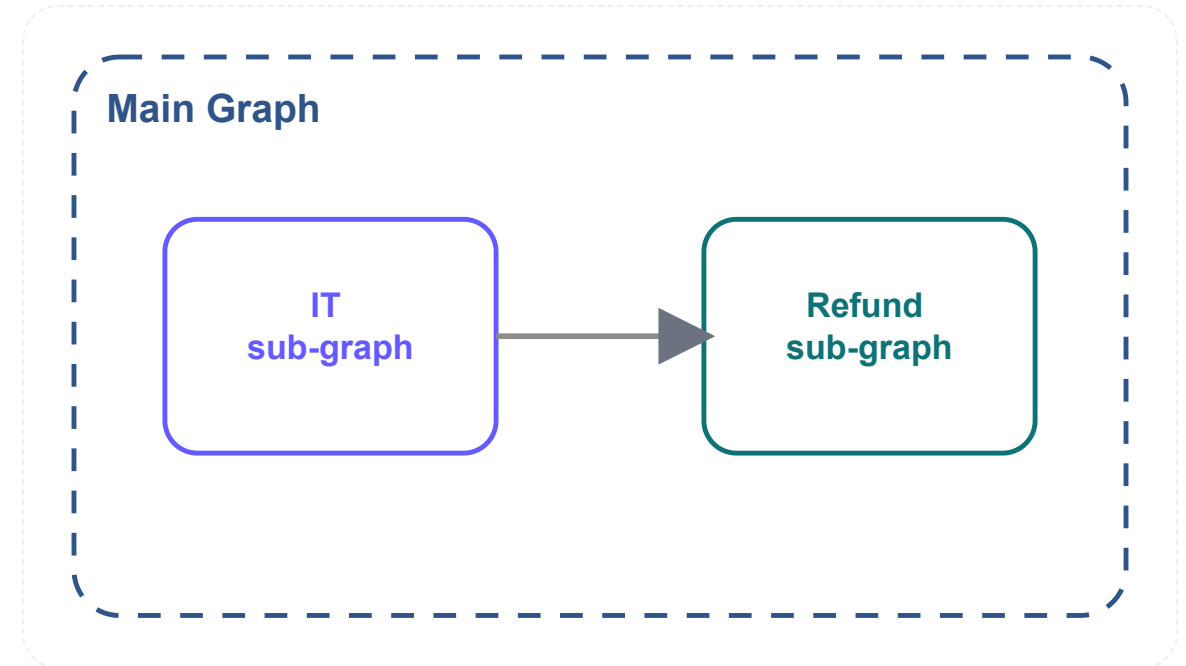


Ý tưởng: LLM A tạo ra kết quả, LLM B (với prompt khắt khe hơn) kiểm tra. Nếu B không hài lòng, nó gửi feedback cho A sửa lại. Giới hạn số lần lặp (ví dụ: tối đa 3 lần) để tránh tốn token.

Sub-graphs: Quản lý độ phức tạp

Chia nhỏ Graph lớn thành các Graph con có thể tái sử dụng

- **Modularity:** Team IT xây dựng `IT\_Graph`, Team Sales xây dựng `Sales\_Graph`.
- **Encapsulation:** Graph lớn chỉ cần biết Input/Output của Graph con, không cần biết chi tiết bên trong.
- **Scalability:** Dễ dàng bảo trì và mở rộng từng phần độc lập.



Hands-on 9 — Supervisor + Workers + MCP + Trace

4 giờ thực hành: refactor Day 08 thành hệ multi-agent quan sát được

1

Refactor graph

supervisor skeleton chạy flow cơ bản

2

Build workers

retrieval · tool/policy · synthesis

3

Thêm MCP

gọi 1 capability thật hoặc mock

4

Trace & docs

compare single vs multi + report cá nhân

Hệ tối thiểu cần build

request → supervisor → 2–3 workers → answer

↳ trace/log cho từng bước route & IO

↳ 1 capability qua MCP

Owner

supervisor

Owner

workers

Owner

MCP + trace

Nhắc mạnh: log chỉ ghi “agent chạy xong” là chưa đủ. Phải thấy đường đi của quyết định.

Hướng dẫn thực hành A — Sprint 1 & 2

Refactor graph trước, rồi mới nhân worker

SPRINT 1 · 60 PHÚT

Refactor artifact Day 08 thành graph/orchestrator

- Supervisor nhận task và route flow cơ bản
- Graph chạy end-to-end ở mức tối thiểu
- Chưa cần tối ưu prompt; cần khung route trước

SPRINT 2 · 60 PHÚT

Tạo 2–3 workers thật rõ vai

- retrieval worker
- tool/policy worker
- synthesis worker

STARTER STRUCTURE

```
graph.py
workers/
  retrieval.py
  policy_tool.py
  synthesis.py
contracts/
  worker_contracts.yaml
artifacts/
  traces/
```

Checklist trước khi sang Sprint 3

- Supervisor route được ít nhất 1 happy path
- Mỗi worker test độc lập được
- Input/output của worker đã chốt

Hướng dẫn thực hành B — Sprint 3 & 4

MCP capability, trace evidence, docs và báo cáo cá nhân

SPRINT 3 · 60 PHÚT

Nối 1 external capability qua MCP

- gọi capability thật hoặc mock
- route đúng worker
- ghi lại lúc nào supervisor chọn MCP

SPRINT 4 · 60 PHÚT

Trace toàn flow + compare single vs multi

- log task, route reason, IO từng worker
- docs/routing_decisions.md
- docs/single_vs_multi_comparison.md

Deliverables nhóm

- supervisor/orchestrator chạy được
- 2–3 workers rõ vai trò
- 1 MCP-connected capability
- trace/log toàn bộ luồng
- 3 docs: system_architecture · routing_decisions · single_vs_multi_comparison

Báo cáo cá nhân — ngắn gọn nhưng đủ ý

1. Tôi phụ trách phần nào?

Nói rõ module, worker, contract hoặc phần trace bạn trực tiếp làm.

2. Tôi đã ra một quyết định gì?

Chọn 1 quyết định route hoặc contract do chính bạn đề xuất và giải thích vì sao.

3. Tôi đã sửa một lỗi gì?

Mô tả ngắn lỗi, cách sửa và bằng chứng trước/sau như log, file, commit hoặc screenshot.

State vs message passing — dùng cái nào khi nào?

Shared state cho orchestration · message contract cho giao việc rõ ràng

TRONG GRAPH

Shared state

- giữ context chung của toàn flow
- hữu ích khi supervisor cần nhìn toàn cục
- phù hợp với route reason, confidence, status

Ví dụ state: task, risk_high, needs_tool, chosen_worker, retrieved_chunks, final_answer

QUA RANH GIỚI WORKER

Message contract

- input/output ổn định để test độc lập
- dễ version hoá và thay worker
- rõ lỗi vì có error format nhất quán

Ví dụ message: task, context, constraints, expected_output, error {code, reason}

Rule of thumb: state để **điều phối cả flow**; message để **giao việc qua boundary**. Trong lab, hai thứ này thường đi cùng nhau chứ không loại trừ nhau.

Route signal & HITL checklist

Supervisor nên route theo tín hiệu quan sát được, không theo cảm giác

Task type

câu hỏi cần retrieval, tool, policy hay synthesis?

Capability needed

có cần external capability hoặc worker chuyên biệt không?

Confidence

bằng chứng hiện có đủ mạnh chưa, có xung đột không?

Side effect / risk

có hành động thật, phê duyệt, ghi dữ liệu hay chạm policy nhạy cảm không?

Khi nào cần HITL?

Low confidence

evidence yếu, route không chắc hoặc output có nhiều giả định

High-risk action

gửi mail, đổi quyền truy cập, cập nhật ticket hoặc side effect thật

Policy conflict

nhiều nguồn nói khác nhau hoặc policy mới vừa đổi version

User-visible escalation

trả lời cuối có thể ảnh hưởng quyết định nghiệp vụ

Rule of thumb: route bằng tín hiệu đo được; đặt HITL ở chỗ **rủi ro cao, confidence thấp** hoặc **có side effect**.

Trace tốt mới giúp học orchestration

Từ output cuối sang reasoning flow quan sát được; cầu nối sang Day 10

```
01 task: "khách hỏi refund sau 7 ngày?"
02 route: retrieval_worker
03 chunks: policy/refund-v4.pdf#p3
04 route: policy_worker
05 rule: refund policy 7 ngày
06 synthesize: answer + cite + fallback
07 final: "Không hoàn tiền sau 7 ngày..."
```

route_reason = policy

confidence = 0.91

1 Trace nên ghi

task · route reason · worker IO · answer cuối

2 Multi-agent không phải để tăng số agent

mục tiêu là hệ rõ vai hơn

3 Cầu nối sang Day 10

data quality + observability giữ hệ thống chạy thống bền



Q&A

Nếu answer đang sai, hệ của bạn thiếu gì nhiều hơn: worker, MCP capability hay trace?